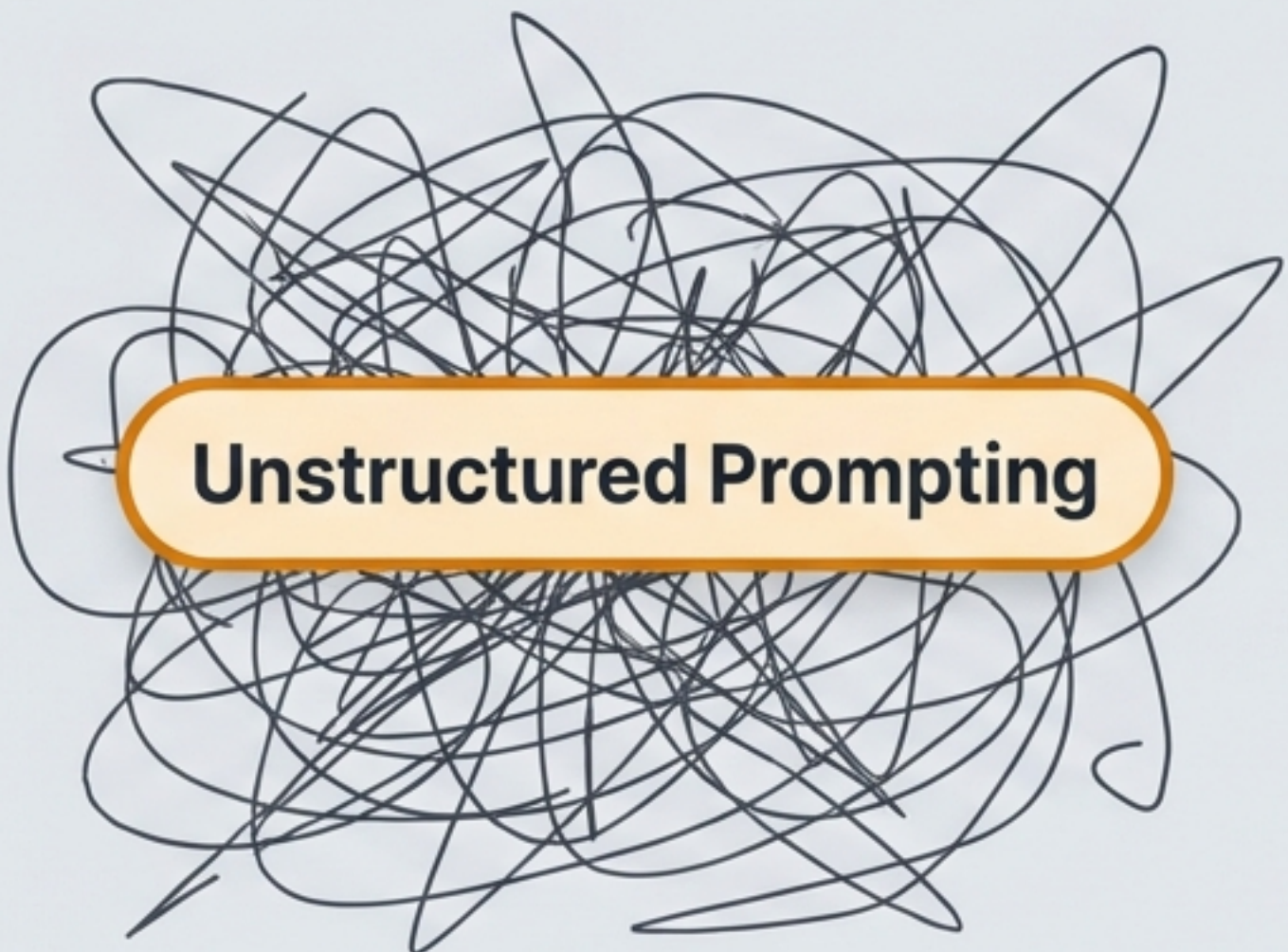
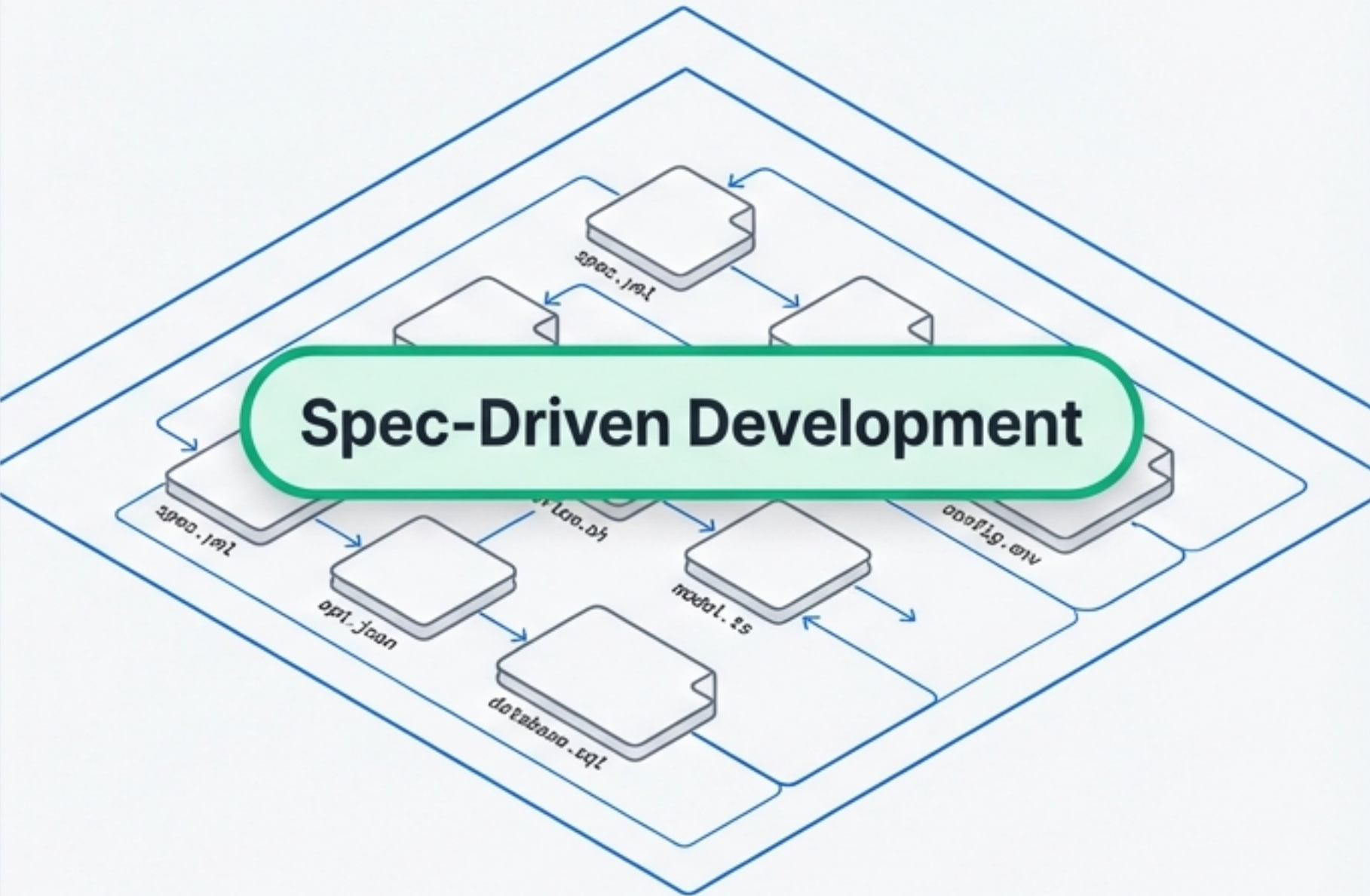


GitHub Spec Kit: From 'Vibe Coding' to Spec-Driven Development

A structured workflow for building scalable software with AI Agents.



Unstructured Prompting



Spec-Driven Development

The diagram illustrates a structured workflow for Spec-Driven Development. It features a central green rounded rectangle labeled 'Spec-Driven Development' surrounded by a blue diamond-shaped frame. Inside the frame, several document icons represent different files: 'spec.json', 'api.json', 'model.ts', 'database.sql', and 'config.env'. Arrows indicate a flow from 'spec.json' to 'api.json', then to 'model.ts', and finally to 'database.sql'. A feedback loop arrow points from 'database.sql' back to 'spec.json'. The 'config.env' file is also shown within the structure.

The Trap of “Vibe Coding”

“Vibe coding” is the act of chatting with an LLM and hoping for the best. It feels magical at first, but lacks precision.

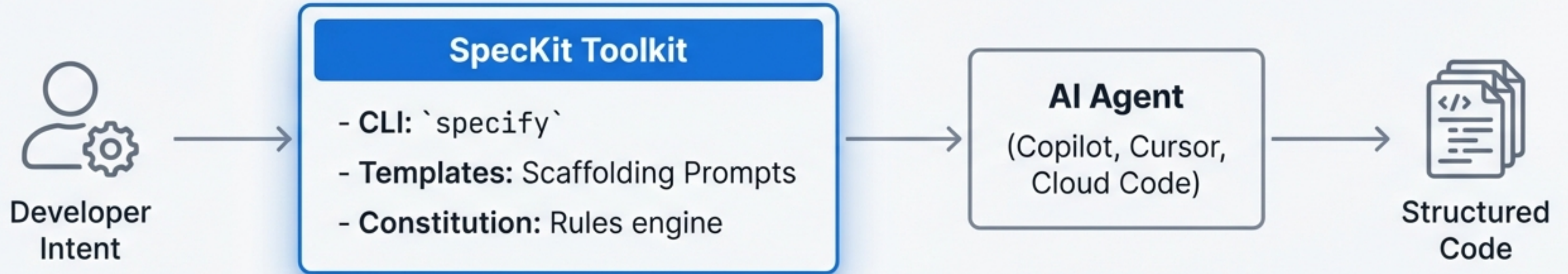
Pain Points:

- ❌ **Imprecision:** The output is rarely exactly what you envisioned.
- ❌ **Rabbit Holes:** You spend hours fixing code that misunderstood the architecture.
- ❌ **Unscalable:** Works for snippets, fails for systems.



Enter SpecKit: A Convenience Layer for AI

SpecKit is an open-source toolkit designed to bridge the gap between human intent and AI execution. It provides the scaffolding needed to turn vague ideas into rigorous software.



The CLI

``specify`` (via ``uvx`` or download)

The Templates

Pre-built prompts and folder structures.

The Agents

Compatible with Copilot, Cursor, and local scripts.

Bootstrapping the Project

1. Initialize

```
uvx specify init  
# or download zip from releases
```

2. Select Agent

Choose your environment (e.g., GitHub Copilot, Cursor).

3. Select Script

Defaults to OS (PowerShell for Windows, Shell for Linux/macOS).

```
user@dev:~/projects $ uvx specify init
```

SPECIFY

? Select your agent:

➤ GitHub Copilot

Cursor

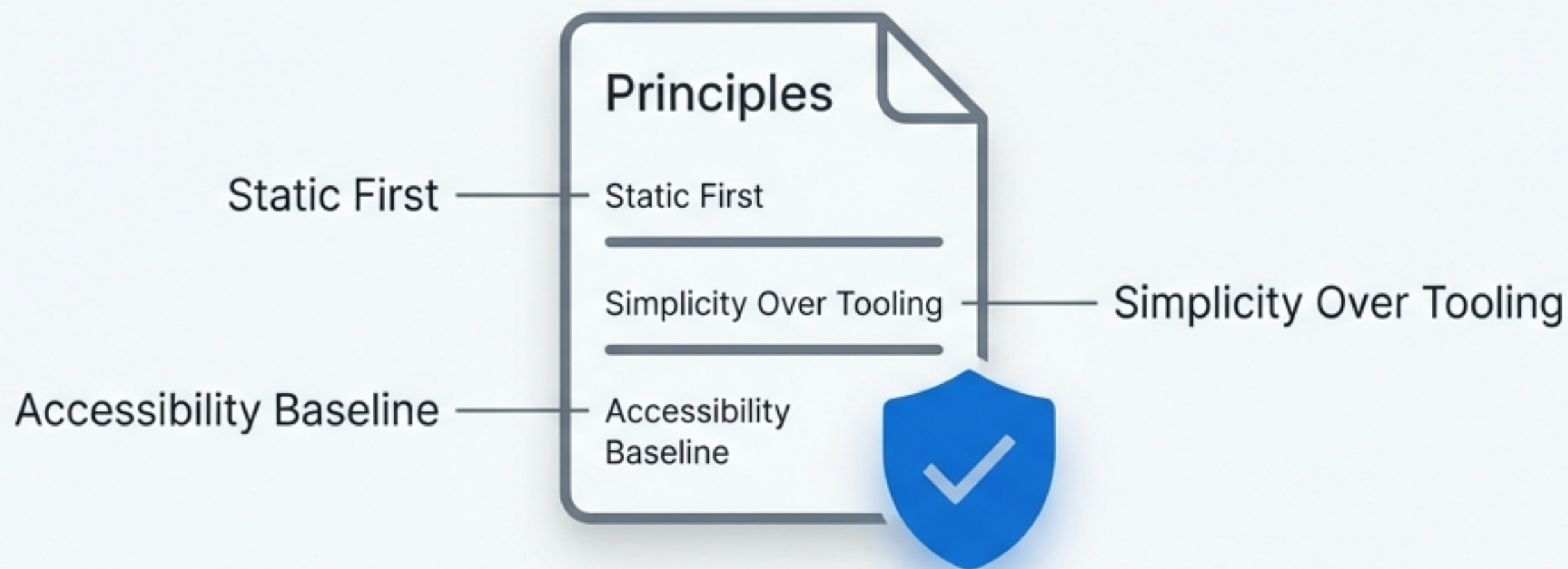
Cloud Code

Google Gemini

Downloading templates...

Phase I: The Constitution

Establishing Non-Negotiable Principles



Before defining *what* to build, you must define *how* you build. The Constitution creates a boundary for the LLM to prevent it from choosing overly complex stacks or ignoring best practices.

Command: `/specify` (updating the constitution)

The Constitution in Action

Example: Defining 'Bare Minimum' for a Podcast Site

memory/constitution.md

memory > constitution.md

```
1 # Project Constitution
2
3 ## Article 1: Static First
4 - No server-side execution.
5 - The site ships HTML/CSS/JS via CDN.
6 - Use Static Site Generation (SSG).
7
8 ## Article 2: Simplicity Over Tooling
9 - Prefer vanilla HTML/CSS or lightweight frameworks.
10 - Avoid complex state management unless necessary.
11
12 ## Article 3: Accessibility Baseline
13 - Must meet WCAG 2.1 AA standards.
14 - Semantic HTML is mandatory.
```

Deployment Constraint

Tech Stack Guardrail

Quality Standard

Phase II: The Specification (The What & Why)

Subhead in the phase, native designs across the product modification requirements but but this regular goals define framework.

The Concept

Define product requirements without technical bias. Focus on the user and the business goals, not the framework.

Command: /specify

The Input Prompt

I am building a modern podcast website.

- Style: Sleek and minimalist.
- Pages: Landing (featured episode), Episodes List, About, FAQ.
- Data: Mocked (no real feeds yet).
- Content: 20 mock episodes.



Do: Describe user flow, visual vibe, pages.



Don't: Mention React, Next.js, or SQL.

The Artifact: spec.md

Automatically generated Product Requirement Document (PRD)

Generated Output: Comprehensive Spec



Make a podcast site.



```
spec.md — Preview

## User Scenarios

- As a listener, I want to play the featured episode directly from the landing page.
- As a new visitor, I want to read about the host on the About page.

## Functional Requirements

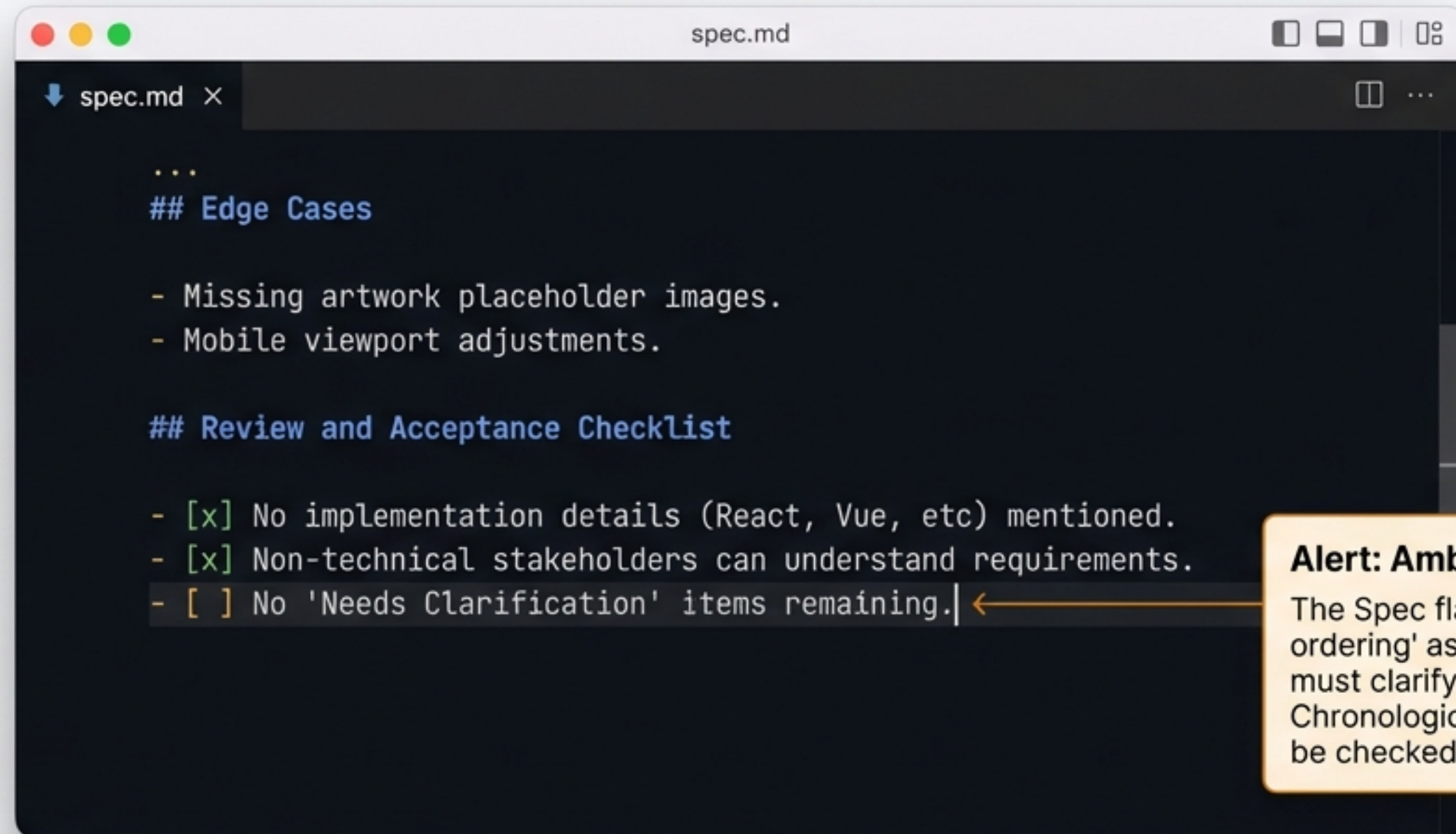
- Landing Page: Hero section with play button, 'Subscribe' links.
- Episodes: Grid layout, pagination for 20 items.

## Edge Cases

- Missing artwork placeholder images.
- Mobile viewport adjustments.
```


The Review & Acceptance Checklist

Human-in-the-loop verification



Goal: Ensure the 'Needs Clarification' list is empty before generating the technical plan.

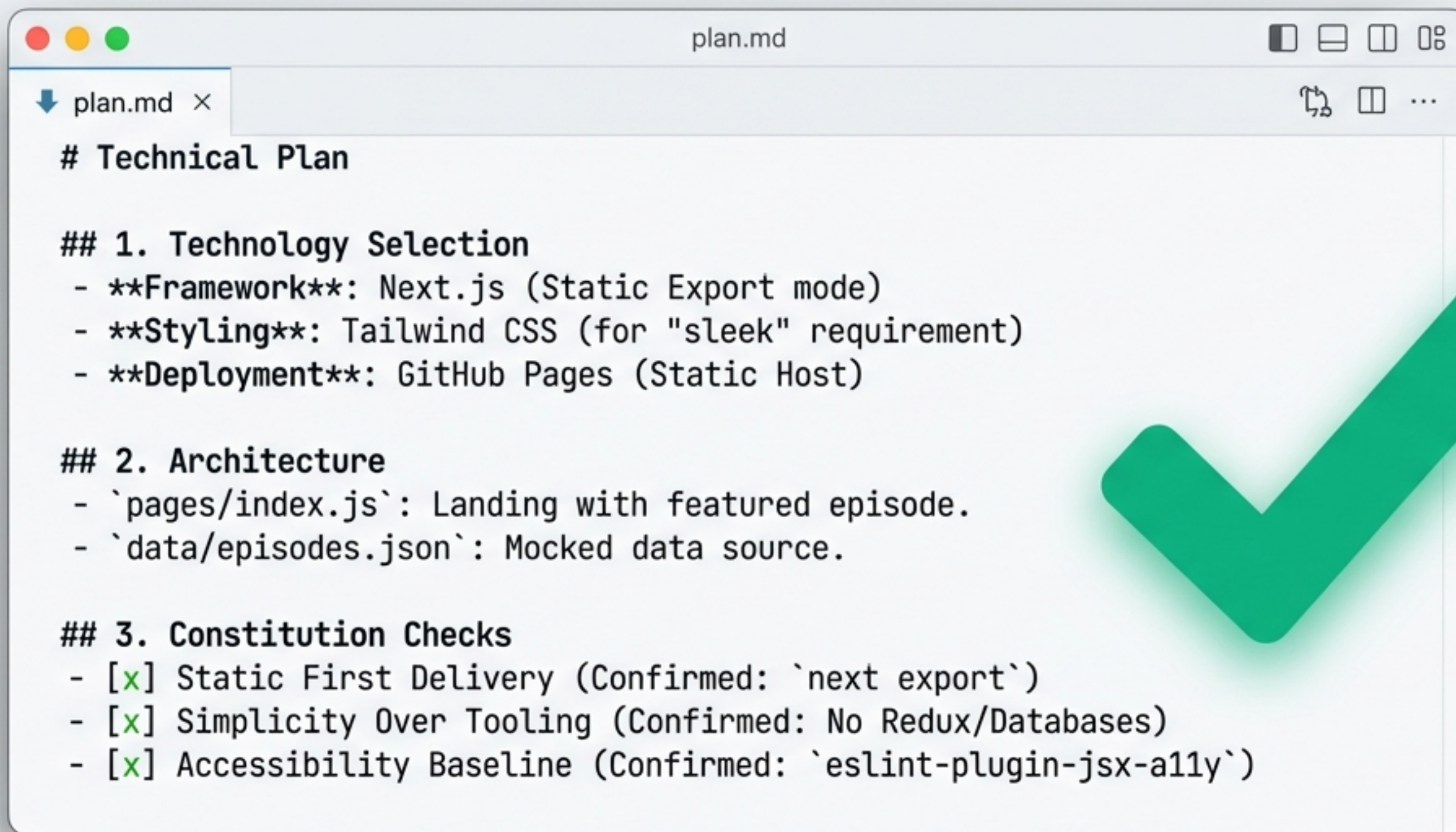
Phase III: The Plan (The Technical How)

Translating Goals into Engineering Tasks



Command: `/plan`

Analyzing the Generated Plan



```
plan.md

# Technical Plan

## 1. Technology Selection
- **Framework**: Next.js (Static Export mode)
- **Styling**: Tailwind CSS (for "sleek" requirement)
- **Deployment**: GitHub Pages (Static Host)

## 2. Architecture
- `pages/index.js`: Landing with featured episode.
- `data/episodes.json`: Mocked data source.

## 3. Constitution Checks
- [x] Static First Delivery (Confirmed: `next export`)
- [x] Simplicity Over Tooling (Confirmed: No Redux/Databases)
- [x] Accessibility Baseline (Confirmed: `eslint-plugin-jsx-a11y`)
```


Phase IV: The Tasks (Breaking It Down)

Sequential Execution Strategy

tasks.md

1. **Project Setup**: Initialize Next.js skeleton and install dependencies.
2. **Test First**: Create failing tests for Landing Page and About Page (TDD).
3. **Core Implementation**: Build components to pass the tests.
4. **Data Layer**: Create `episodes.json` mock data.
5. **Refinement**: Run Lighthouse check and fix accessibility issues.

Sequential Execution

Command: `/tasks`

Strategy: Selecting the Right Model for the Phase

The Architect (Planning)



Model: GPT-5 / High-Reasoning

- Thoughtful, slow, considers constraints, better at structural scaffolding.

Use for:

`Constitution`
`Spec`
`Plan`

The Builder (Execution)



Model: Claude 3.5 Sonnet / High-Coding

- Creative, fast, superior syntax generation, unbeatable for implementation.

Use for:

`Tasks`
`Code Generation`

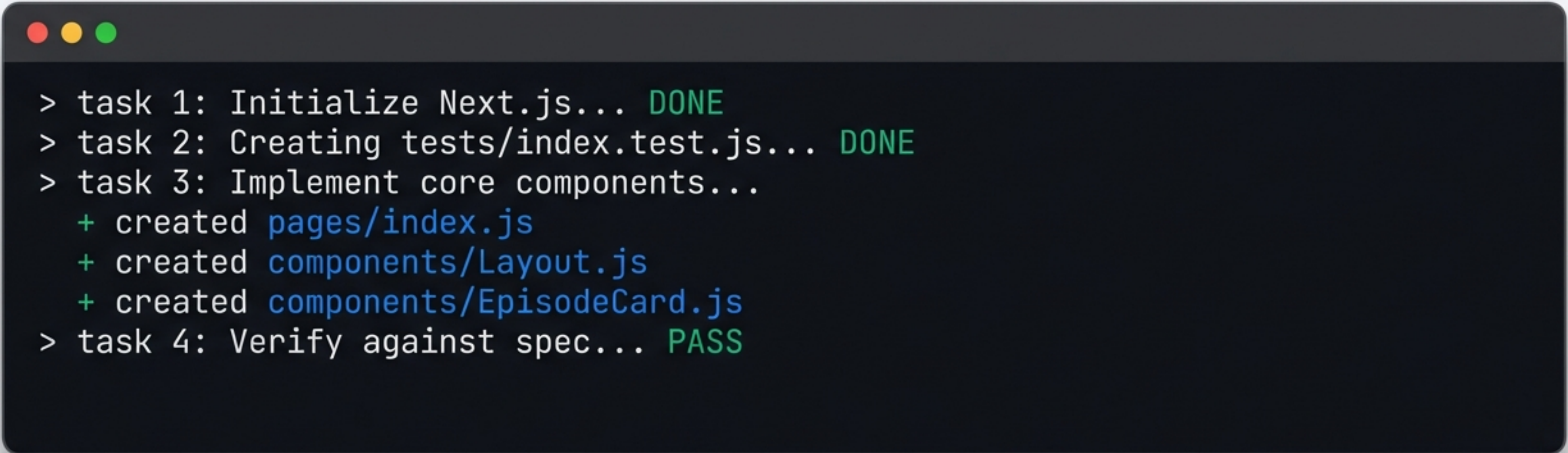
Reasoning

Coding



Phase V: Execution & Iteration

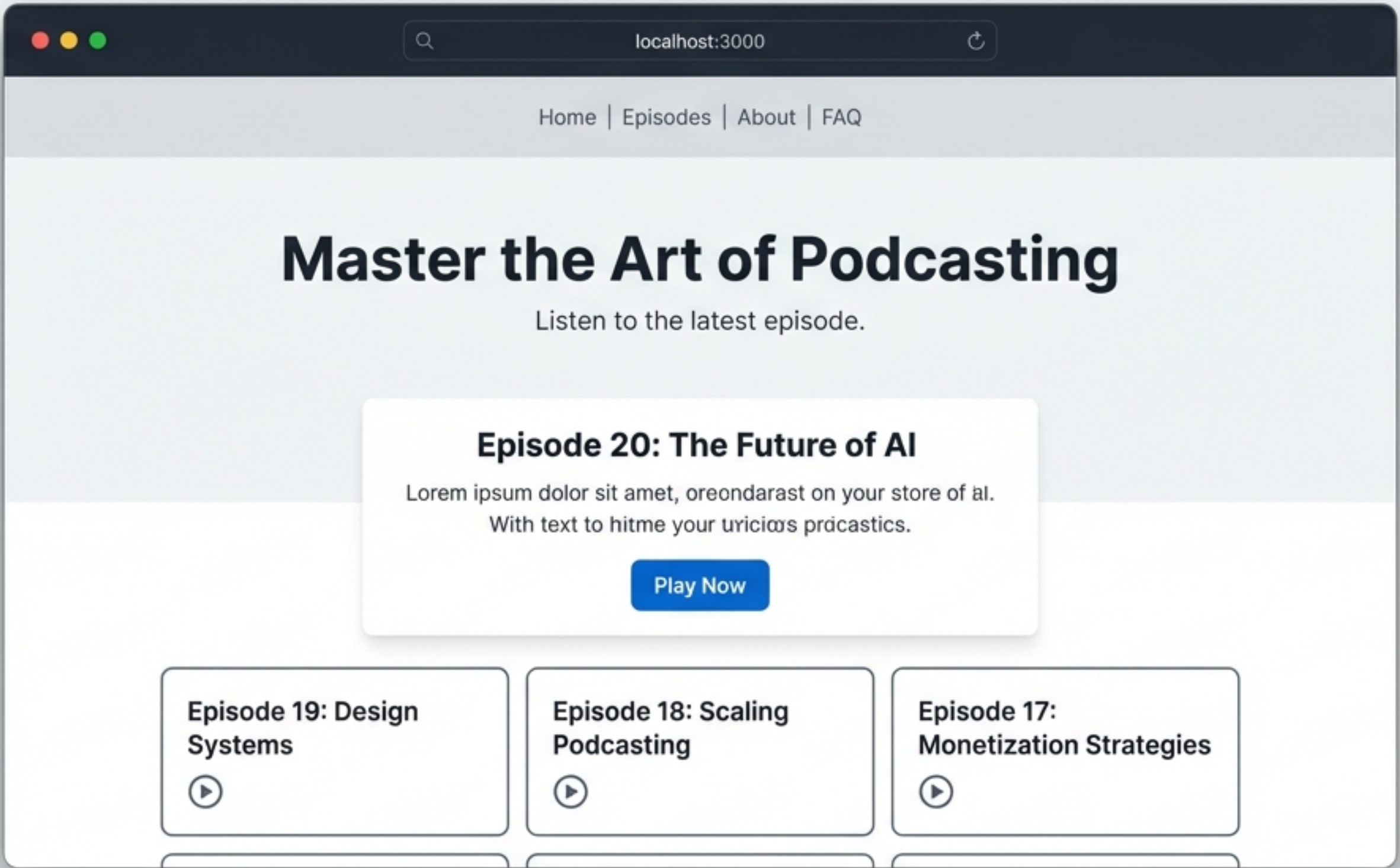
The Agent implements the task list.



```
> task 1: Initialize Next.js... DONE
> task 2: Creating tests/index.test.js... DONE
> task 3: Implement core components...
+ created pages/index.js
+ created components/Layout.js
+ created components/EpisodeCard.js
> task 4: Verify against spec... PASS
```

Human Role: Watch the terminal, approve file creations, handle build errors.

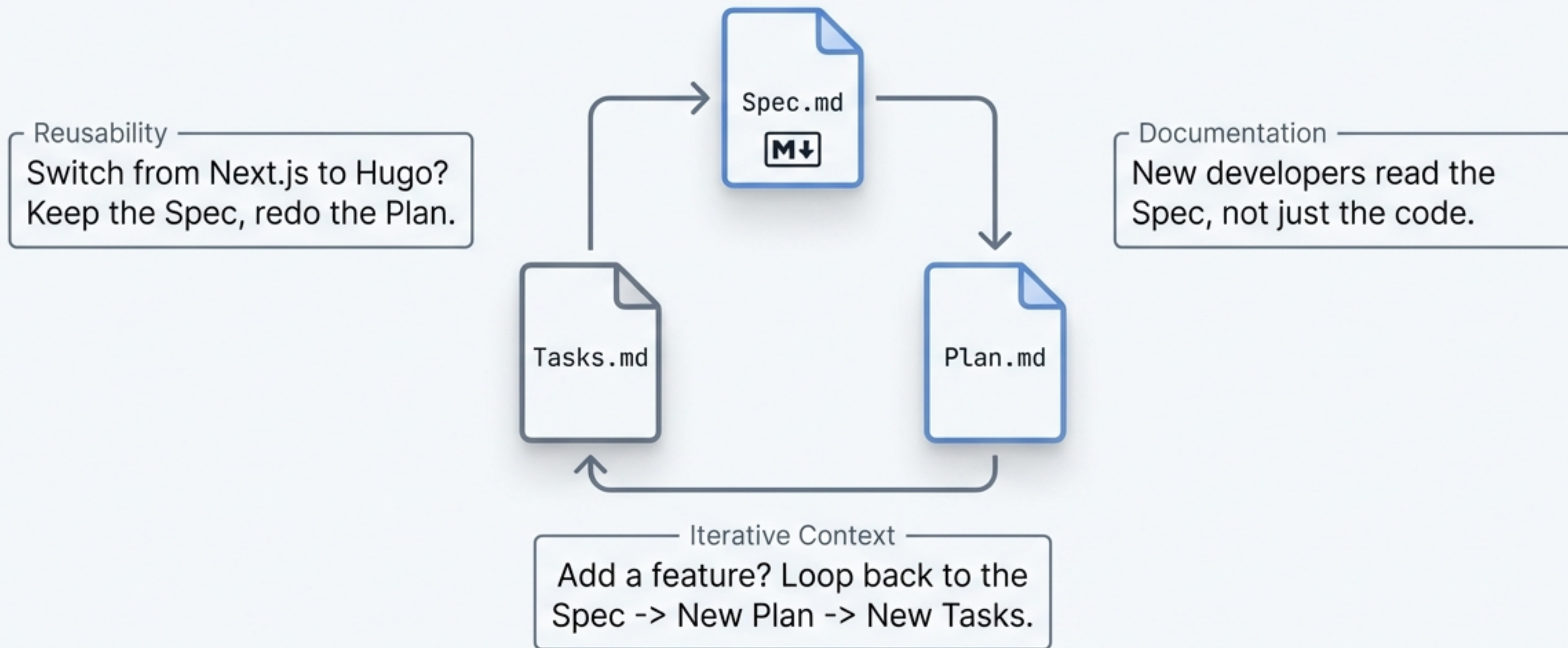
The Result: A 'Sleek' Podcast Site



Functional prototype created from spec.md without manual coding.

The True Value is the Artifacts

Code is brittle. Context is durable.



“The artifacts become that piece of executable context that you can pass to your team.”

Start Your Experiment



`github.com/github/spec-kit`

Call to Action:

- Download the CLI.
- Run the Podcast Site demo.
- Submit feedback (Issues welcome!).

This is an open-source experiment. Break it, fix it, and help define the future of software development.

